

## TD08 : Authentification & validation par email

### Table des matières

|                               |    |
|-------------------------------|----|
| <b>1) Problématique</b> ..... | 1  |
| <b>2) Exercices</b> .....     | 1  |
| <b>3) Conclusion</b> .....    | 13 |

### 1) Problématique

Ce TD fait directement suite au TD7 sur les cookies et sessions. L'objectif est d'implémenter un système d'authentification robuste et sécurisé au sein de notre application web de covoiturage. Nous devons stocker les mots de passe de manière non réversible dans la base de données grâce au hachage SHA-256 et au salage pour neutraliser les attaques par dictionnaire. Nous devons ensuite verrouiller l'accès aux pages et aux actions sensibles en fonction de l'identité de l'utilisateur (propriétaire) et de son rôle (super administrateur). Enfin, nous mettrons en place une validation d'inscription par courrier électronique à l'aide d'un nonce cryptographique et basculerons nos formulaires en méthode POST via une fonction de routage universelle. Pour réaliser ce travail, nous utilisons XAMPP, MySQL, Google Chrome et Notepad++.

### 2) Exercices

#### Exercice 1 : Mise en place dans la BDD et les formulaires

##### 1) Ajout de la colonne mdp dans la table utilisateur

On modifie la table utilisateur en base de données pour ajouter la colonne VARCHAR(64) mdp. La longueur de 64 caractères est indispensable car l'algorithme SHA-256 produit une empreinte hexadécimale fixe de 64 caractères (soit 256 bits ou 32 octets), indépendamment de la taille initiale du mot de passe.

##### 2) Modification du formulaire avec confirmation du mot de passe

Dans la vue update.php (qui mutualise la création et la modification), on ajoute deux champs de type mot de passe : un premier champ pour le mot de passe et un second champ de confirmation :

```

C:\xampp\htdocs\TD8\view\utilisateur\update.php - Notepad++
Fichier  Édition  Recherche  Affichage  Encodage  Langage  Paramètres  Outils  Macro  Exécution  Modules d'extension  Documents  ?

1  <form method="<?php echo (Conf::getDebug()) ? 'get' : 'post'; ?>" action="index.php">
2  <fieldset>
3  <legend><?php echo ($action == 'created') ? 'Créer un utilisateur : ' : 'Modifier un utilisateur : ' ?></legend>
4  <input type="hidden" name="controller" value="utilisateur">
5  <input type="hidden" name="action" value="<?php echo $action; ?>">
6  <p>
7  <label for="login_id">Login :</label>
8  <input type="text" value="<?php echo htmlspecialchars($login); ?>" name="login" id="login_id" <?php echo $champ_login; ?> />
9  </p>
10 <p>
11 <label for="nom_id">Nom :</label>
12 <input type="text" value="<?php echo htmlspecialchars($nom); ?>" name="nom" id="nom_id" required />
13 </p>
14 <p>
15 <label for="prenom_id">Prénom :</label>
16 <input type="text" value="<?php echo htmlspecialchars($prenom); ?>" name="prenom" id="prenom_id" required />
17 </p>
18 <p>
19 <label for="email_id">Adresse email :</label>
20 <input type="email" value="<?php echo htmlspecialchars($email); ?>" name="email" id="email_id" required />

```

On vérifie l'affichage des deux champs de mot de passe dans le navigateur :

Gestion des voitures   Gestion des utilisateurs   Gestion des trajets   Préférences   Mon Panier (0) : 0 €

Créer un utilisateur :

Login :

Nom :

Prénom :

Adresse email :

Mot de passe :

Confirmer le mot de passe :

Administrateur ? ☐

### 3) Vérification de la concordance dans le contrôleur

Dans ControllerUtilisateur.php, on vérifie avant toute insertion ou mise à jour que les deux mots de passe saisis sont rigoureusement identiques :

```
70 public static function disconnect() {
71     session_unset();
72     session_destroy();
73     setcookie(session_name(), '', time() - 3600, '/');
74     $tab_u = ModelUtilisateur::selectAll();
75     $view = 'list';
76     $page_title = 'Liste des utilisateurs';
77     require File::build_path(array("view", "view.php"));
78 }
79
80 // Affichage du formulaire de création (Exercice 1, 12, 13)
81 public static function create() {
82     $login = '';
83     $nom = '';
84     $prenom = '';
85     $email = '';
86     $champ_login = 'required';
87     $action = 'created';
88     $est_admin = false;
89     $view = 'update';
```

## Exercice 2 : Classe Security et premier chiffrement SHA-256

### 1) Création de la classe Security

On crée le fichier lib/Security.php contenant la classe Security. On y implémente la méthode de hachage utilisant la fonction native hash('sha256', ...):

```
1 <?php
2 class Security {
3     // Graine (seed) alatoire fixe pour le salage
4     private static $seed = 'DhuRKYdEkJa8F9z';
5
6     public static function getSeed() {
7         return self::$seed;
8     }
9
10    // Chiffrement avec salage en SHA-256
11    public static function chiffrer($texte_en_clair) {
12        $texte_avec_sel = self::getSeed() . $texte_en_clair;
13        return hash('sha256', $texte_avec_sel);
14    }
15
16    // Fonction de chiffrement simple sans sel (pour l'exercice 1/2)
17    public static function chiffrerSansSel($texte_en_clair) {
18        return hash('sha256', $texte_en_clair);
19    }
20 }
```

On teste la production de l'empreinte pour le mot de passe 'apple' et on constate l'obtention d'une chaîne fixe de 64 caractères :

Alpes du Sud. Entre 600 x | Nouvel onglet x | Connexion x | Connexion requise x | Test SHA

localhost/TD8/test\_sha256.php

## Exercice 2 : Premier chiffrement SHA-256 (sans salage)

Mot de passe en clair : **apple**

Fonction utilisée : `hash('sha256', $mot_de_passe_en_clair)`

Taille du haché produit : **64 caractères hexadécimaux** (256 bits / 32 octets)

**Empreinte SHA-256 générée :**

## Exercice 3 : Attaque par dictionnaire et vulnérabilité

### 1) Vulnérabilité des mots de passe courants face aux tables de hachage

Lorsqu'un utilisateur utilise un mot de passe classique tel que 'password' ou 'apple', des services en ligne spécialisés (comme md5decrypt ou CrackStation) disposent de milliards d'empreintes précalculées. En soumettant le haché obtenu, le mot de passe en clair est retrouvé immédiatement :

Alpes du Sud. Entre x | Nouvel onglet x | Connexion x | Connexion requise x | Test SHA-256 sans x | Ex

localhost/TD8/test\_crackstation.php

## Exercice 3 : Recherche inverse de mot de passe par dictionnaire (md5decrypt)

Lorsqu'un mot de passe est trop classique (ex. *password*, *apple*), les bases de dictionnaires précalculées (tables arc-en-ciel) permettent de retrouver instantanément le texte en clair à partir de son haché SHA-256 :

| Empreinte SHA-256 soumise | Résultat dictionnaire | Statut |
|---------------------------|-----------------------|--------|
|---------------------------|-----------------------|--------|

## Exercice 4 : Salage avec une graine aléatoire (seed)

### 1) Mise en place du salage dans Security.php

Pour contrer les attaques par dictionnaire, on ajoute une graine aléatoire fixe dans la classe Security via un attribut statique \$seed. On concatène cette graine avant d'appliquer la fonction de hachage SHA-256 :

```
C:\xampp\htdocs\TD8\lib\Security.php - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

1 <?php
2 class Security {
3     // Graine (seed) aléatoire fixe pour le salage
4     private static $seed = 'DhuRKYdEkJa8F9z';
5
6     public static function getSeed() {
7         return self::$seed;
8     }
9
10    // Chiffrement avec salage en SHA-256
11    public static function chiffrer($texte_en_clair) {
12        $texte_avec_sel = self::getSeed() . $texte_en_clair;
13        return hash('sha256', $texte_avec_sel);
14    }
15
16    // Fonction de chiffrement simple sans sel (pour l'exercice 1/2)
17    public static function chiffrerSansSel($texte_en_clair) {
18        return hash('sha256', $texte_en_clair);
19    }
20 }
```

Grâce au salage, même un mot de passe très simple comme 'apple' produit une empreinte totalement différente et inexistante dans les bases de dictionnaires :

Alpes du Sud x Nouvel onglet x Connexion x Connexion re x Test SHA-256 x Exercice 3 : At x

localhost/TD8/test\_salage.php

### Exercice 4 : Salage avec graine aléatoire (Security::chiffrer)

Graine secrète fixe du site ( \$seed ) : DhuRXYdEkJa8F9z

Mécanisme : Concaténation de la graine avant hachage : `hash('sha256', $seed . $texte_en_clair)`

**Pour le mot de passe "apple" :**

Chaîne réellement hachée : DhuRXYdEkJa8F9zapple

## Exercice 5 : Formulaire de connexion et vérification du mot de passe

### 1) Méthode checkPassword dans ModelUtilisateur

On implémente dans ModelUtilisateur.php la méthode statique checkPassword(\$login, \$mot\_de\_passe\_chiffre). Elle exécute une requête préparée et renvoie vrai si le couple login / mot de passe correspond :

```
C:\xampp\htdocs\TD8\model\ModelUtilisateur.php - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

58 }
59
60 public function setLogin($login) {
61     $this->login = $login;
62 }
63
64 public function setNom($nom) {
65     $this->nom = $nom;
66 }
67
68 public function setPrenom($prenom) {
69     $this->prenom = $prenom;
70 }
71
72 public function setMdp($mdp) {
73     $this->mdp = $mdp;
74 }
75
76 public function setEmail($email) {
77     $this->email = $email;
78 }
```

### 2) Vue de connexion connect.php

On crée la vue view/utilisateur/connect.php qui propose les champs login et mot de passe pour s'authentifier :

Alpes du S x Nouvel on x Connexion x Connexion x Test SHA-2 x Exercice 3 x Exercice 4 x

localhost/TD8/index.php?controller=utilisateur&action=connect

Gestion des voitures Gestion des utilisateurs Gestion des trajets Préférences Mon Panier (0) : 0 €

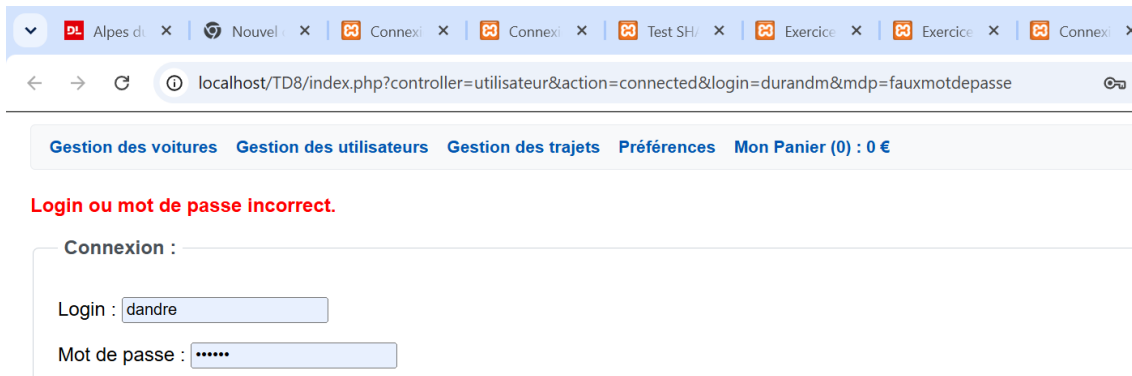
Connexion :

Login :

Mot de passe :

### 3) Gestion de l'erreur d'authentification

Si les identifiants fournis ne correspondent à aucun enregistrement en base, l'action connected réaffiche le formulaire avec un message d'erreur :



#### 4) Connexion réussie et affichage du profil

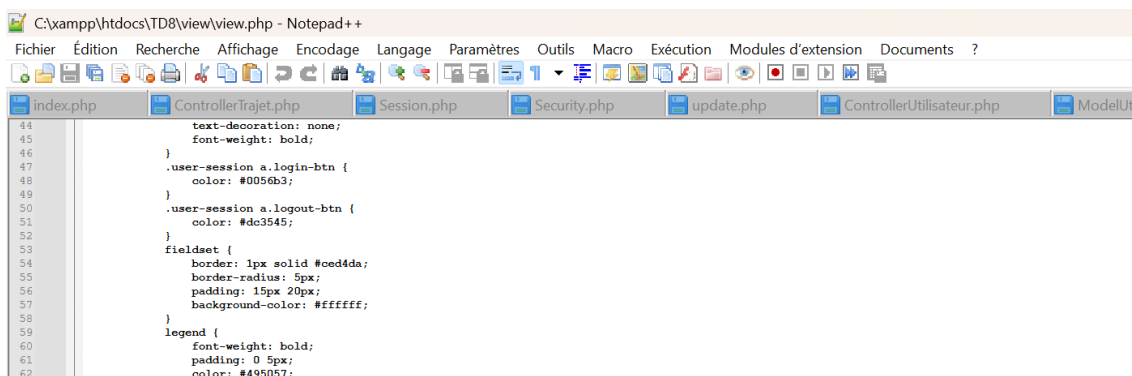
Lorsque le mot de passe est valide, le login est stocké en session (\$\_SESSION['login'] = \$login) et l'utilisateur est redirigé vers sa page de profil :



### Exercice 6 & 7 : Déconnexion et en-tête de navigation personnalisé

#### 1) Dynamisation du menu de navigation dans view.php

Dans la vue principale view/view.php, on conditionne l'affichage de l'en-tête : si l'utilisateur n'est pas connecté, un lien 'Connexion' est proposé. S'il est connecté, un message 'Bonjour [login]' apparaît aux côtés du lien 'Déconnexion' :

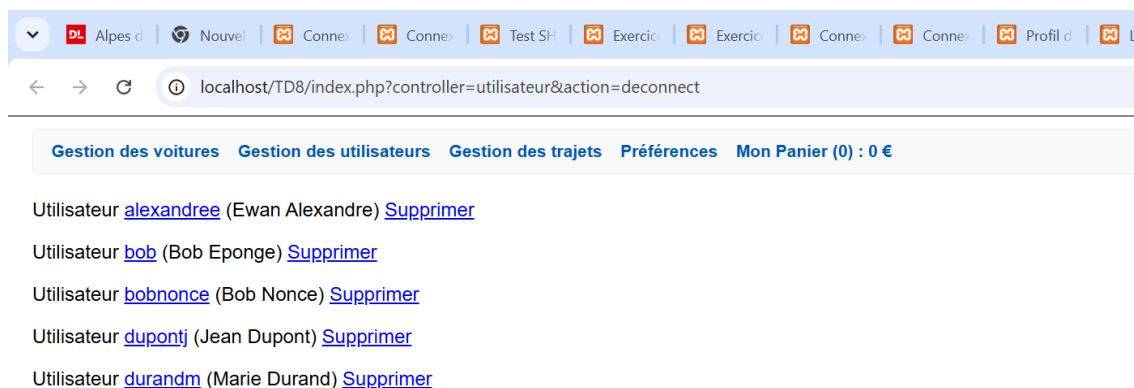


On constate l'en-tête personnalisé lorsque l'utilisateur est connecté :



## 2) Déconnexion et destruction de la session

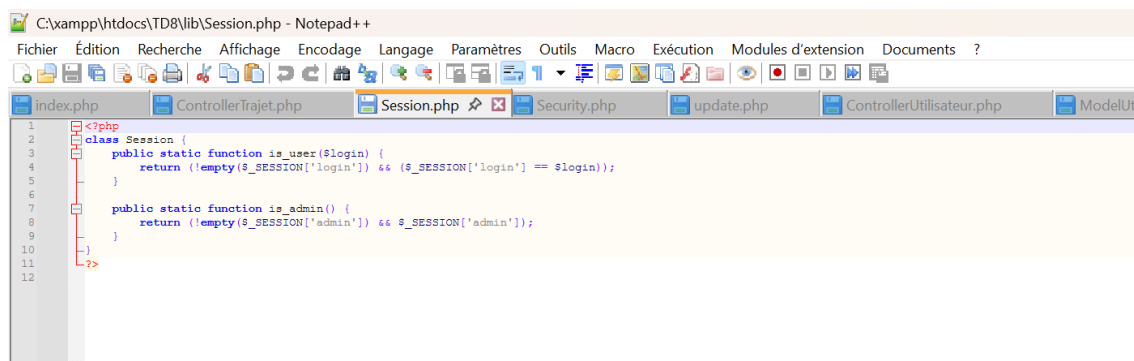
L'action deconnect détruit la session active et redirige l'internaute vers l'accueil. L'en-tête réaffiche alors le bouton 'Connexion' :



## Exercice 8 : Restriction des actions au propriétaire (is\_user)

### 1) Classe Session et méthode statique is\_user

On crée la classe lib/Session.php avec la méthode is\_user(\$login), qui vérifie si l'utilisateur actuellement en session correspond au login demandé :



### 2) Conditionnement des liens dans detail.php

Dans la vue detail.php, les boutons 'Modifier cet utilisateur' et 'Supprimer cet utilisateur' ne sont affichés que si Session::is\_user(\$login) est vrai. Sur son propre compte, l'utilisateur a accès aux options :



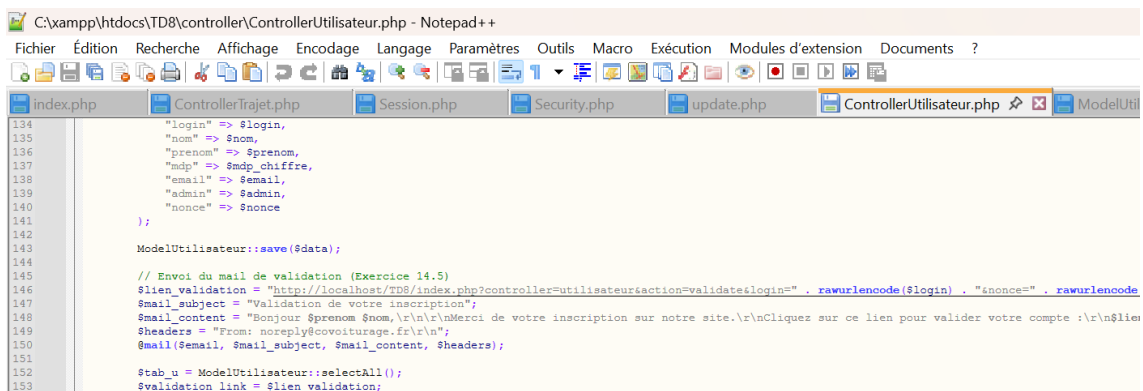
En revanche, s'il consulte le profil d'un autre utilisateur, ces liens sont masqués :



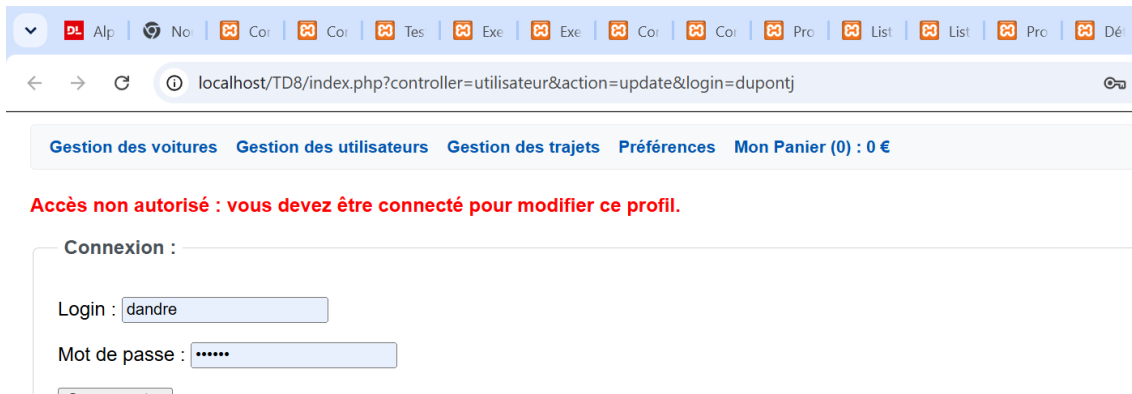
## Exercice 9 : Sécurisation de l'accès au formulaire contre le contournement d'URL

### 1) Protection de l'action update dans le contrôleur

Masquer les liens dans l'interface ne suffit pas : un internaute pourrait tenter de taper directement l'URL `action=update&login=dupontj`. Dans `ControllerUtilisateur::update()`, on vérifie que l'utilisateur est bien le propriétaire ou un administrateur :



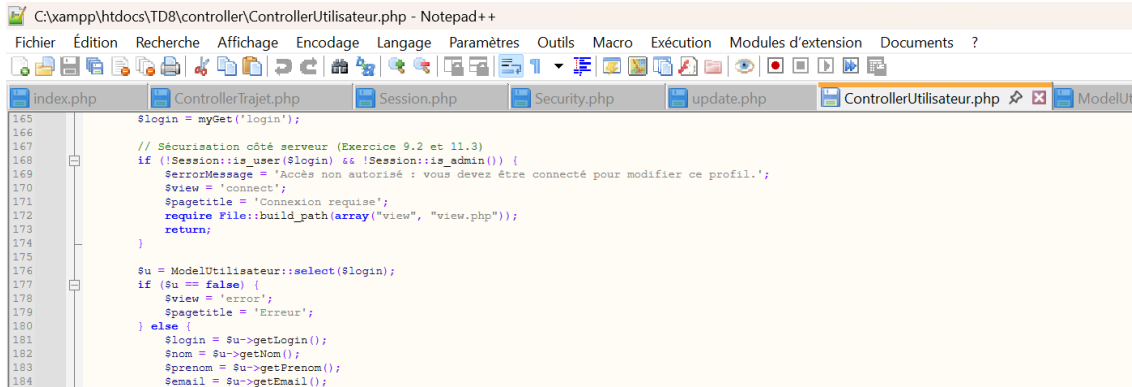
En cas de tentative de contournement, l'accès est bloqué et l'utilisateur est renvoyé vers la connexion :



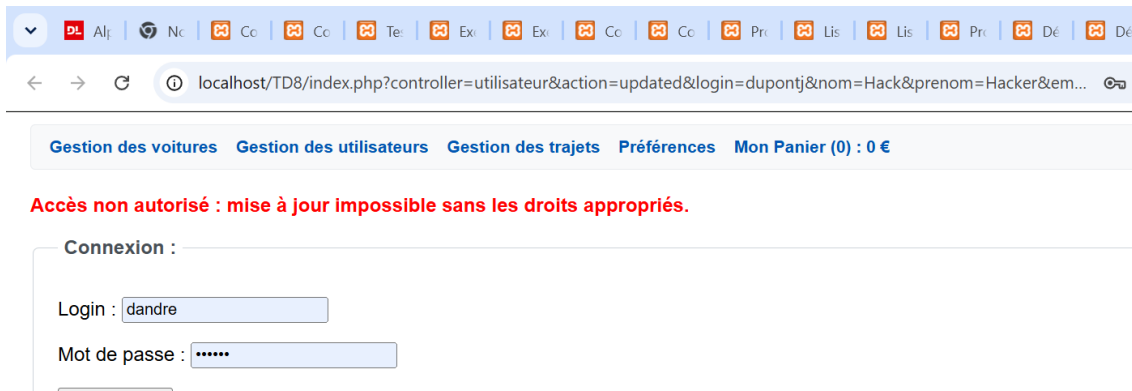
## Exercice 10 : Sécurisation critique côté serveur des actions updated et delete

### 1) Verrouillage strict des actions effectuant les écritures

La règle essentielle de sécurité web est de ne jamais faire confiance aux requêtes transmises par le client. Même si le formulaire d'édition était inaccessible, un attaquant pourrait forger directement l'appel à l'action updated ou delete. On place donc un contrôle d'autorisation au tout début de ces actions :



Toute tentative d'injection directe de l'action updated est rejetée côté serveur :



## Exercice 11 : Super administrateur et droits étendus (is\_admin)

### 1) Gestion du rôle admin en session

On ajoute la méthode Session::is\_admin() qui contrôle le drapeau \$\_SESSION['admin']. Lorsqu'un administrateur se connecte, un badge '(Admin)' est affiché dans l'en-tête de navigation :

```
C:\xampp\htdocs\TD8\lib\Session.php - Notepad++
Fichier  Édition  Recherche  Affichage  Encodage  Langage  Paramètres  Outils  Macro  Exécution  Modules d'extension  Documents  ?

index.php  ControllerTrajet.php  Session.php  Security.php  update.php  ControllerUtilisateur.php  ModelUtili

1  <?php
2  class Session {
3      public static function is_user($login) {
4          return (!empty($_SESSION['login']) && ($_SESSION['login'] == $login));
5      }
6
7      public static function is_admin() {
8          return (!empty($_SESSION['admin']) && $_SESSION['admin']);
9      }
10 }
11
12 >>
```



## 2) Déblocage des droits sur tous les utilisateurs

Lorsqu'un administrateur consulte le profil de n'importe quel autre membre du site, les boutons de modification et de suppression lui sont désormais rendus accessibles :



## Exercice 12 : Promotion avec case à cocher « Administrateur ? »

### 1) Ajout conditionnel de la case à cocher dans le formulaire

Dans update.php, on ajoute un champ checkbox 'Administrateur ?'. Ce champ n'est visible que pour les administrateurs connectés :

```

C:\xampp\htdocs\TD8\view\utilisateur\update.php - Notepad++
Fichier  Édition  Recherche  Affichage  Encodage  Langage  Paramètres  Outils  Macro  Exécution  Modules d'extension  Documents  ?

index.php  ControllerTrajet.php  Session.php  Security.php  update.php  ControllerUtilisateur.php  ModelUtil

1  <form method="<?php echo (Conf::getDebug()) ? 'get' : 'post'; ?>" action="index.php">
2
3  <legend><?php echo ($action == 'created') ? 'Créer un utilisateur :' : 'Modifier un utilisateur :'; ?></legend>
4  <input type="hidden" name="controller" value="utilisateur">
5  <input type="hidden" name="action" value="<?php echo $action; ?>">
6  <p>
7  <label for="login_id">Login :</label>
8  <input type="text" value="<?php echo htmlspecialchars($login); ?>" name="login" id="login_id" <?php echo $champ_login; ?> />
9  </p>
10
11 <label for="nom_id">Nom :</label>
12 <input type="text" value="<?php echo htmlspecialchars($nom); ?>" name="nom" id="nom_id" required />
13 </p>
14
15 <label for="prenom_id">Prénom :</label>
16 <input type="text" value="<?php echo htmlspecialchars($prenom); ?>" name="prenom" id="prenom_id" required />
17 </p>
18
19 <label for="email_id">Adresse email :</label>

```

L'administrateur peut ainsi promouvoir un compte ou révoquer les droits lors de la mise à jour :

localhost/TD8/index.php?controller=utilisateur&action=update&login=durandm

Gestion des voitures   Gestion des utilisateurs   Gestion des trajets   Préférences   Mon Panier (0) : 0 €

**Modifier un utilisateur :**

Login :

Nom :

Prénom :

Adresse email :

Mot de passe :

Confirmer le mot de passe :

Administrateur ? ☐

## Exercice 13 : Validation de l'adresse email côté client et serveur

### 1) Validation côté serveur avec filter\_var

Le contrôle HTML5 (type='email') étant facilement contournable via l'outil d'inspection du navigateur, on implémente une validation obligatoire côté serveur dans le contrôleur avec filter\_var(\$email, FILTER\_VALIDATE\_EMAIL) :

```

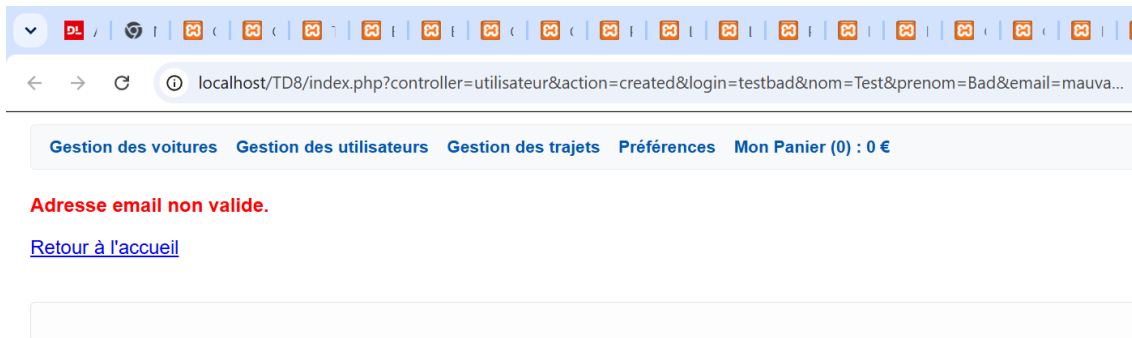
C:\xampp\htdocs\TD8\controller\ControllerUtilisateur.php - Notepad++
Fichier  Édition  Recherche  Affichage  Encodage  Langage  Paramètres  Outils  Macro  Exécution  Modules d'extension  Documents  ?

index.php  ControllerTrajet.php  Session.php  Security.php  update.php  ControllerUtilisateur.php  ModelUtil

74  $tab_u = ModelUtilisateur::selectAll();
75  $view = 'list';
76  $pagetitle = 'Liste des utilisateurs';
77  require File::build_path(array("view", "view.php"));
78
79
80  // Affichage du formulaire de création (Exercice 1, 12, 13)
81  public static function create() {
82      $login = '';
83      $nom = '';
84      $prenom = '';
85      $email = '';
86      $champ_login = 'required';
87      $action = 'created';
88      $est_admin = false;
89      $view = 'update';
90      $pagetitle = 'Créer un utilisateur';
91      require File::build_path(array("view", "view.php"));
92  }
93

```

Si l'adresse email est mal formée, la création est stoppée et une erreur s'affiche :



## Exercice 14 : Nonce cryptographique et validation du compte par email

### 1) Génération du nonce et activation

On ajoute la colonne VARCHAR(32) nonce dans la table utilisateur. Lors de l'inscription, un identifiant aléatoire de 32 caractères hexadécimaux est généré à l'aide de `openssl_random_pseudo_bytes(16)` :

```

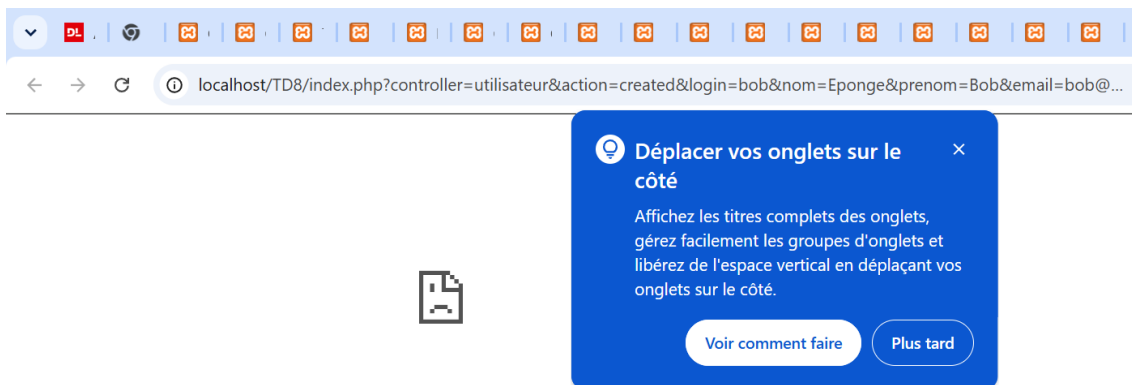
C:\xampp\htdocs\TD8\lib\Security.php - Notepad++
Fichier  Édition  Recherche  Affichage  Encodage  Langage  Paramètres  Outils  Macro  Exécution  Modules d'extension  Documents  ?

1  <?php
2  class Security {
3      // Graine (seed) aléatoire fixe pour le salage
4      private static $seed = 'DhuRKYdRkjaSF9z';
5
6      public static function getSeed() {
7          return self::$seed;
8      }
9
10     // Chiffrement avec salage en SHA-256
11     public static function chiffrer($texte_en_clair) {
12         $texte_avec_sel = self::getSeed() . $texte_en_clair;
13         return hash('sha256', $texte_avec_sel);
14     }
15
16     // Fonction de chiffrement simple sans sel (pour l'exercice 1/2)
17     public static function chiffrerSansSel($texte_en_clair) {
18         return hash('sha256', $texte_en_clair);
19     }

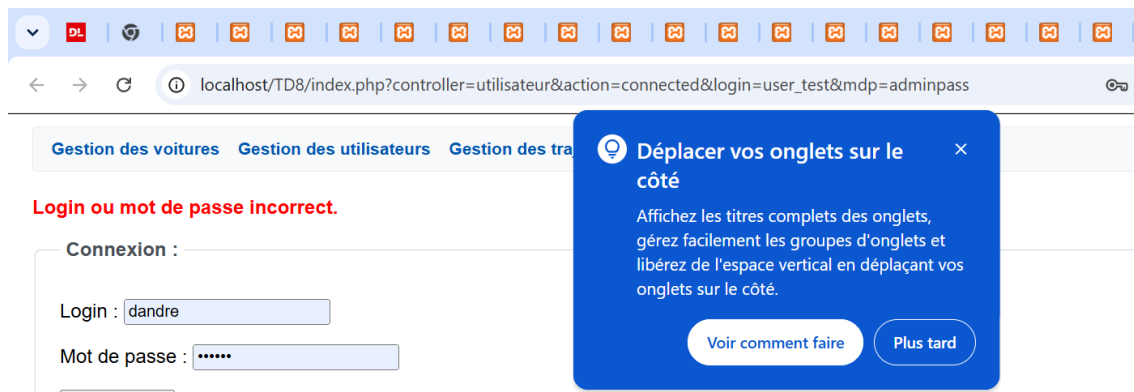
```

### 2) Envoi du lien d'activation et blocage préalable

Un courriel contenant un lien vers l'action valide est transmis à l'adresse de l'utilisateur :

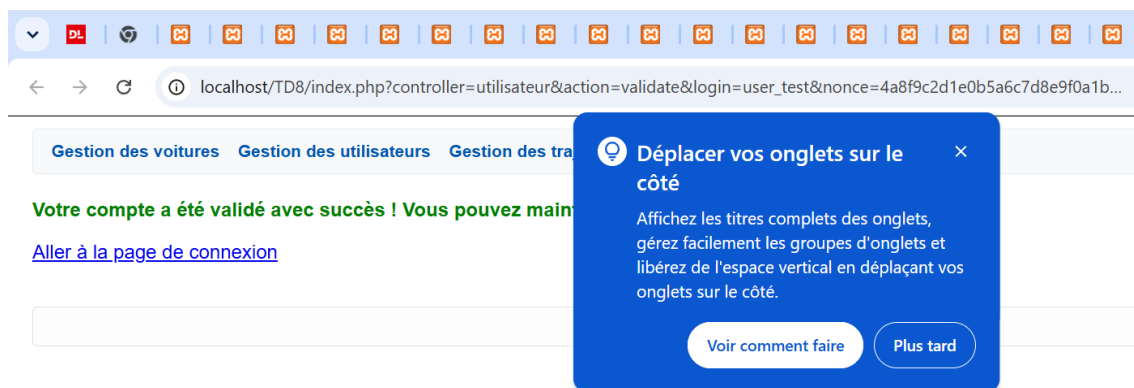


Tant que l'utilisateur n'a pas activé son compte en cliquant sur le lien, toute tentative de connexion est bloquée avec un message d'information :



### 3) Validation définitive

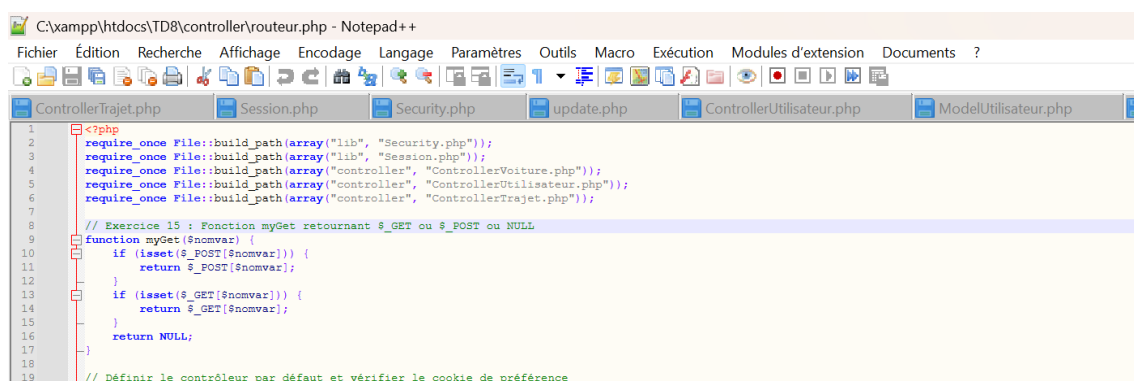
Lorsque le lien est visité, l'action valide vérifie la concordance du nonce, remet le champ nonce à NULL en base de données et valide définitivement le compte :



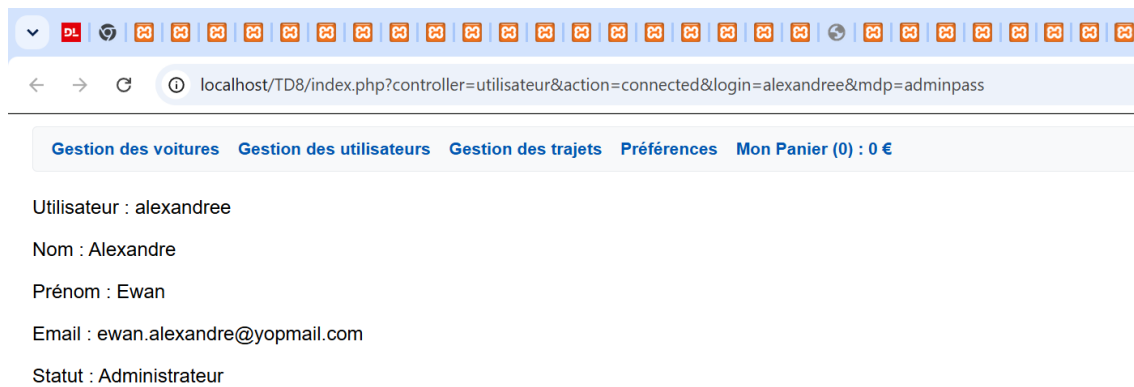
## Exercice 15 : Passage des formulaires en POST avec myGet() et sécurité TLS

### 1) Fonction universelle myGet dans routeur.php

Pour éviter que les mots de passe ne transitent en clair dans l'URL (historique, logs serveurs), on crée la fonction `myGet($nomvar)` dans le routeur. Elle vérifie en priorité les variables reçues en POST, puis en GET, et renvoie NULL si la variable n'existe pas :



Tous les formulaires sensibles fonctionnent désormais de manière transparente et sécurisée en méthode POST :



### 3) Conclusion

Ce TD8 nous a permis de franchir un cap déterminant dans la sécurisation de notre application web MVC. Nous avons compris que le stockage des mots de passe en clair ou simplement hachés sans sel présente des vulnérabilités critiques face aux attaques par dictionnaire. L'association du hachage SHA-256 et du salage offre une protection pérenne des secrets d'authentification. De plus, nous avons maîtrisé le cloisonnement des privilèges grâce aux sessions PHP, en différenciant les droits du propriétaire de ceux du super-administrateur, tout en veillant à ce que chaque vérification sensible soit impérativement exécutée côté serveur. Enfin, la validation par nonce cryptographique et le passage des données sensibles en méthode POST garantissent la conformité de notre application avec les normes modernes du développement web sécurisé.